

WIPRO TRAINING PROGRAM

Capstone Project Report

Customer Engagement Platform

A Web-Based Customer and Ticket Management System

Submitted By

Saumya Singh

Technology Stack

ASP.NET Core MVC | SQL Server | EF Core | JWT | Swagger

Wipro Ltd. | June 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	5
2. Project Overview.....	6
2.1 Application Entry Point	6
2.2 Core Data Model.....	6
2.3 Architecture Summary	6
3. Business Problem Statement	7
3.1 Problems Identified.....	7
3.2 How This Application Addresses These Problems	7
4. Project Objectives	8
5. Functional Requirements.....	9
6. Non-Functional Requirements.....	10
7. System Architecture.....	11
7.1 Architecture Layers	11
7.2 Request Flow.....	12
8. Data Flow Diagram.....	13
8.1 Level 0 - Context Diagram	13
8.2 Level 1 - Main Processes	14
9. Database Design.....	15
9.1 Customer Table	15
9.2 Ticket Table	15
9.3 ASP.NET Identity Tables	15
10. Entity Relationship Diagram.....	17
11. Technology Stack	18
12. Frontend and Backend Connectivity.....	19
12.1 Standard MVC Server-Side Rendering	19
12.2 AJAX Partial View Updates.....	19
12.3 REST API Endpoints	19
13.1 ASP.NET Identity	19
13.2 Roles.....	19
13.3 Route Protection.....	20
13.4 Login and Role Redirect	20
14.1 Step-by-Step Flow	20
14.2 Token Contents.....	21
15. Customer Management Module.....	22
15.1 Features	22

15.2 Business Rule: Active Ticket Check	22
15.3 Validation	22
16. Ticket Management Module.....	23
16.1 Features	23
16.2 Status Workflow.....	23
17. Dashboard Module	24
17.1 Stat Cards	24
17.2 Tickets Needing Assignment	24
17.3 Service Efficiency.....	25
18.1 Filter Controls.....	25
18.2 Technical Implementation	26
18.3 Why This Matters.....	26
19. Repository Pattern Implementation	27
19.1 Structure	27
19.2 Benefits	27
19.3 Registration	27
20. Dependency Injection	28
20.1 What Is Registered	28
20.2 Constructor Injection	28
20.3 Service Lifetime.....	28
22. Design Patterns Used	29
23. Web API Documentation	30
23.1 Request and Response Format	31
24.1 Swagger UI	31
24.2 JWT Authorization.....	31
24.3 Configuration	32
26. Validation and Security Measures	33
26.1 Input Validation	33
26.2 Password Security	33
26.3 SQL Injection Prevention.....	33
26.4 JWT Token Security	33
26.5 Route Protection	33
27. Testing and Verification	34
27.1 Unit Testing	34
27.2 Role-Based Access Testing	34
27.3 API Testing via Swagger	34
27.4 Database Verification.....	35
27.5 CRUD End-to-End Testing.....	35

28.1 Dockerfile	36
28.2 GitHub Actions Workflow	36
30.1 Assumptions Made	38
30.2 Known Limitations	38
31. Best Practices Followed	39
32. Future Scope	40
33. Conclusion	41

1. Executive Summary

The Customer Engagement Platform is a web application built as the Wipro training capstone project. The goal was to create a working system that organizations can use to manage customer records and support ticket requests through a structured, secure platform.

The application provides two separate user experiences. Customers get a self-service portal where they can raise support tickets, track progress, and view their ticket history. Support staff get a management dashboard with ticket statistics, the ability to assign agents, update statuses, and manage customer accounts.

The system is built using ASP.NET Core 8 MVC for the frontend, ASP.NET Core Web API for the REST layer, and SQL Server with Entity Framework Core for the database. ASP.NET Identity handles user registration and authentication, and JWT tokens secure the API endpoints. The project also includes Swagger documentation, AJAX-based live filtering, a Dockerfile, and a GitHub Actions CI/CD workflow.

This report covers the system design, database structure, module-level implementation, security approach, testing results, and future development plans.

2. Project Overview

The Customer Engagement Platform is a multi-layer web application that uses role-based access to separate customer functionality from staff management functionality.

2.1 Application Entry Point

When a user visits the application, they see a home page with two entry points: the Customer Portal and the Staff Dashboard. The two roles are completely isolated. A customer trying to access a staff route is redirected, and a staff user cannot access customer portal routes.

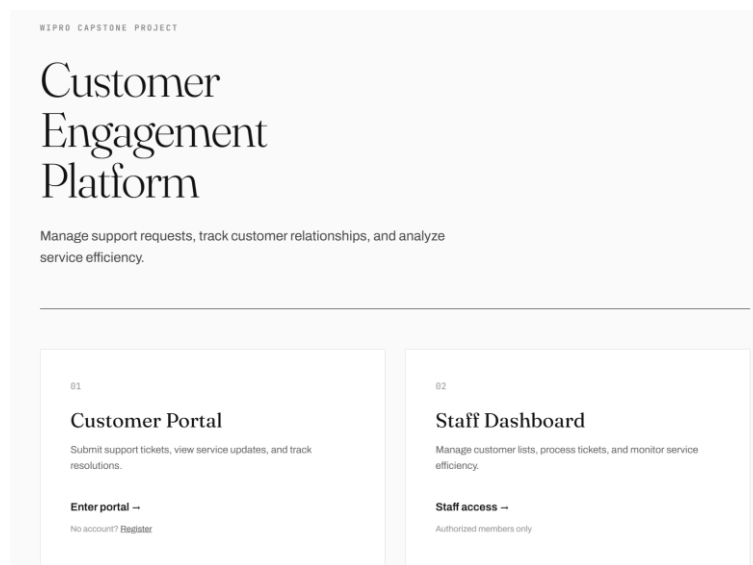


Figure 1: Application landing page showing the two entry points

2.2 Core Data Model

The system maintains two main entities: Customers and Tickets. Each Customer can have multiple Tickets. Each Ticket has a status (Open, In Progress, Resolved), a priority (Low, Medium, High), and can be assigned to a staff agent.

2.3 Architecture Summary

The application follows a layered architecture. The MVC presentation layer handles user requests and renders pages. A separate REST API layer exposes JWT-secured data endpoints. A service layer applies business rules. A repository layer abstracts all database access. Entity Framework Core handles all SQL Server communication.

3. Business Problem Statement

Many organizations still manage customer support through emails and spreadsheets. This creates problems that grow worse as the number of customers and tickets increases.

3.1 Problems Identified

- Support tickets raised by customers through email get lost or overlooked. There is no central place to see all open requests.
- There is no visibility into which tickets are assigned, which are open, and which have been resolved.
- Customer information is scattered across different files, making it hard to quickly find details for a specific customer.
- Without role-based access, there is no separation between what customers should see and what staff manage.
- Reporting on support performance requires manual effort, such as counting emails or rows in a spreadsheet.

3.2 How This Application Addresses These Problems

The Customer Engagement Platform centralizes all of this into one web application. Customers have their own secure portal. Staff have a dashboard that shows all tickets and customers. Ticket status is tracked through a defined workflow. The dashboard provides automatic statistics without manual counting.

4. Project Objectives

The following objectives guided the development of the project:

1. Build a secure authentication system using ASP.NET Identity supporting Customer and Staff roles.
2. Create a Customer Management module for staff to view, add, edit, and delete customer records with business rule enforcement.
3. Create a Ticket Management module supporting the full lifecycle from creation through assignment to resolution.
4. Build a Staff Dashboard displaying key metrics: total customers, total tickets, open, resolved, and unassigned tickets.
5. Implement a Customer Portal where customers can raise tickets and track their own requests.
6. Expose a JWT-secured REST API for customer and ticket data, documented through Swagger.
7. Implement AJAX-based filtering on the tickets page for search and filter without page reloads.
8. Apply the Repository Pattern, Dependency Injection, and SOLID principles throughout the codebase.
9. Include DevOps assets: a Dockerfile for containerization and a GitHub Actions workflow for CI/CD.

5. Functional Requirements

ID	Requirement	Role	Priority
FR-01	User registration with name, email, and password	Both	High
FR-02	Login with role-based redirect to Customer Portal or Dashboard	Both	High
FR-03	View list of all customers	Staff	High
FR-04	Add a new customer record	Staff	High
FR-05	Edit existing customer details	Staff	Medium
FR-06	Delete a customer (only if no active tickets exist)	Staff	Medium
FR-07	View all tickets with search and filter controls	Staff	High
FR-08	Create a new ticket on behalf of a customer	Staff	High
FR-09	Edit ticket details and update status	Staff	High
FR-10	Assign a ticket to a staff agent	Staff	High
FR-11	Delete a ticket	Staff	Low
FR-12	Customer can raise a new support ticket	Customer	High
FR-13	Customer can view their own tickets only	Customer	High
FR-14	Dashboard shows ticket and customer statistics	Staff	High
FR-15	AJAX-based search and filter without page reload	Staff	Medium
FR-16	POST /api/auth/login returns JWT token	API	High
FR-17	GET /api/customers and GET /api/customers/{id}	API	High
FR-18	GET /api/tickets and GET /api/tickets/{id}	API	High
FR-19	Swagger UI with JWT Authorize button	API	Medium

6. Non-Functional Requirements

Category	Requirement
Security	All pages require authentication. Passwords are hashed by ASP.NET Identity. JWT tokens are signed and time-limited.
Authorization	Role-based access enforced on every route. Customers cannot reach staff routes and vice versa.
Performance	AJAX filtering avoids full page reloads. EF Core queries apply WHERE filters at the database level.
Maintainability	Repository Pattern and DI allow components to be replaced or mocked independently.
Usability	Bootstrap 5 UI with consistent navigation and responsive layout for both portal and dashboard.
Portability	Dockerfile provided to run the application without a local .NET setup.
Testability	Service and repository layers are unit-tested with xUnit and Moq. API tested via Swagger.
Data Integrity	EF Core code-first migrations maintain schema consistency. FK constraints prevent orphaned records.

7. System Architecture

The application is built on a layered architecture where each layer has a single clear responsibility and communicates only with adjacent layers.

7.1 Architecture Layers

Layer	Technology	Responsibility
Presentation	ASP.NET Core MVC (Controllers, Views, ViewModels)	Receives HTTP requests, renders Razor views, returns responses to the browser.
REST API	ASP.NET Core Web API (Auth, Customers, Tickets)	Exposes JSON endpoints, validates JWT tokens, returns structured API responses.
Business / Service	C# service classes (TicketService, CustomerService)	Applies business rules and coordinates between controllers and repositories.
Data Access	Repository classes (ITicketRepository, ICustomerRepository)	Abstracts all EF Core queries behind interfaces.
ORM	Entity Framework Core 8 (ApplicationDbContext)	Maps C# entity classes to SQL Server tables and handles migrations.
Database	SQL Server (LocalDB)	Stores all application data including Identity tables.

Customer Engagement Platform – System Architecture

Layered ASP.NET Core MVC Application

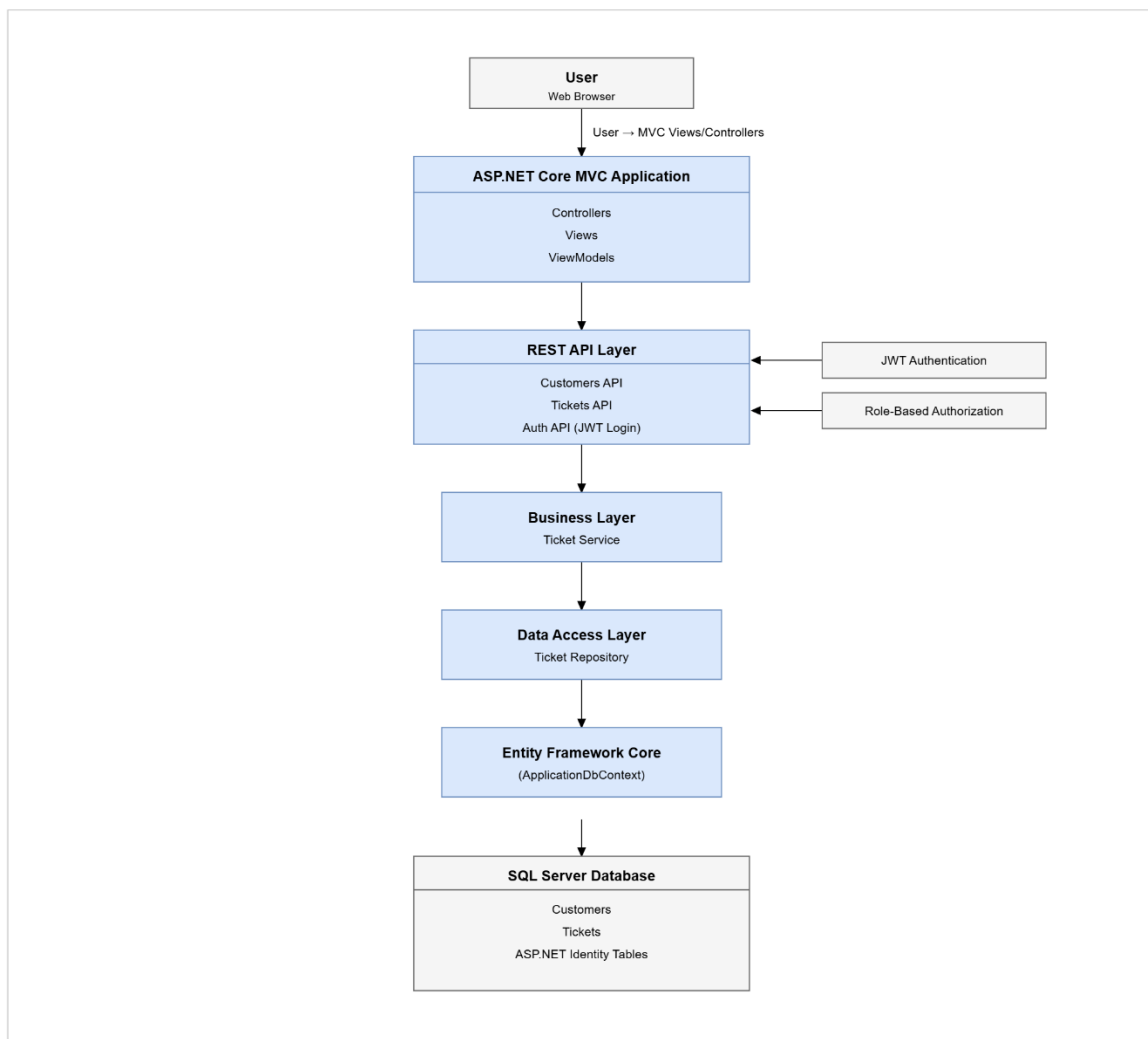


Figure 2: System Architecture Diagram

7.2 Request Flow

A browser request hits the MVC Controller. The Controller calls a Service method. The Service calls the Repository. The Repository queries the database using EF Core. The result travels back up through the same layers and the View renders the HTML response.

For API requests, the same service and repository layers are used. The Web API controller validates the JWT Bearer token before processing the request and returns JSON instead of a rendered view.

8. Data Flow Diagram

The Data Flow Diagram describes how data moves between users, the application, and the database.

8.1 Level 0 - Context Diagram

External entities interacting with the system:

- Customer: Sends login credentials and ticket data. Receives ticket status and history.
- Staff User: Sends management operations. Receives dashboard statistics and data.
- SQL Server Database: The central data store. Receives write operations, returns query results.

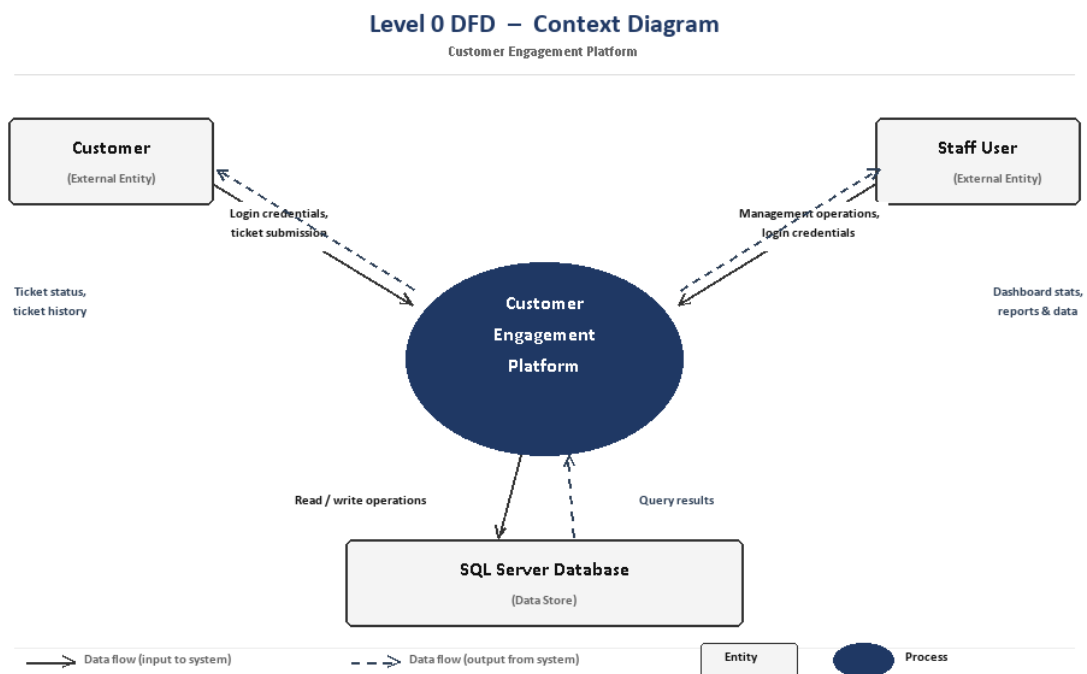
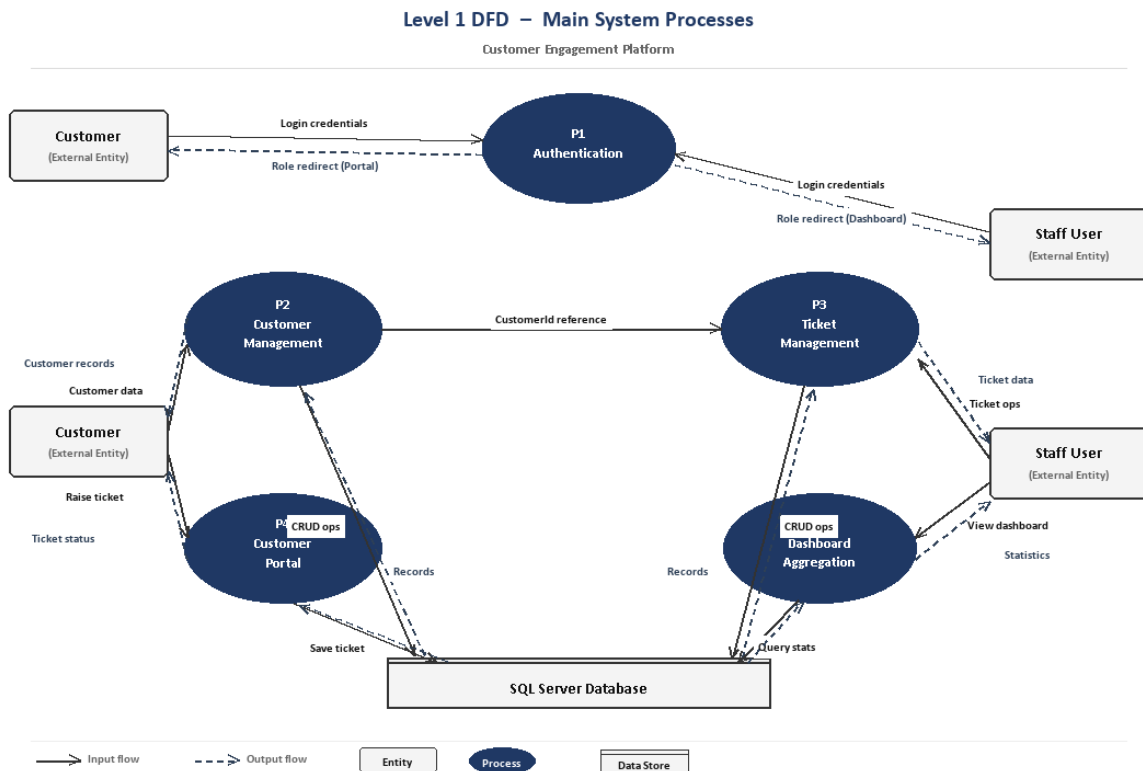


Figure 3: Level 0 DFD - Context Diagram

8.2 Level 1 - Main Processes

Process	Input	Output
Authentication	Email and password from login form	Session cookie (MVC) or JWT token (API)
Customer Management	Customer data from staff forms	Created / updated / deleted customer records
Ticket Management	Ticket data, status updates, agent assignments	Created / updated ticket records
Customer Portal	Ticket form submission from customer	New ticket record, customer ticket list view
Dashboard Aggregation	Database query results	Calculated statistics displayed on the dashboard



9. Database Design

Figure 4: Level 1 DFD - Main Processes

The application uses SQL Server with Entity Framework Core in Code-First mode. All schema changes are managed through EF Core migrations. Running `dotnet ef database update` applies all pending migrations.

9.1 Customer Table

Column	Data Type	Constraints	Description
CustomerId	INT	PK, Identity	Auto-incremented primary key
FirstName	NVARCHAR(100)	NOT NULL	Customer first name
LastName	NVARCHAR(100)	NOT NULL	Customer last name
Email	NVARCHAR(256)	NOT NULL, Unique	Customer email address
Phone	NVARCHAR(20)	NULL	Contact phone number
Address	NVARCHAR(500)	NULL	Customer address
CreatedAt	DATETIME2	NOT NULL	Record creation timestamp
UserId	NVARCHAR(450)	FK to AspNetUsers, NULL	Links to ASP.NET Identity user account

9.2 Ticket Table

Column	Data Type	Constraints	Description
TicketId	INT	PK, Identity	Auto-incremented primary key
Title	NVARCHAR(200)	NOT NULL	Short description of the issue
Description	NVARCHAR(MAX)	NULL	Full issue description
Status	NVARCHAR(50)	NOT NULL	Open / In Progress / Resolved
Priority	NVARCHAR(20)	NOT NULL	Low / Medium / High
CreatedAt	DATETIME2	NOT NULL	Ticket creation timestamp
CustomerId	INT	FK to Customers, NOT NULL	The customer who raised the ticket
AssignedTo	NVARCHAR(256)	NULL	Staff agent name assigned to the ticket

9.3 ASP.NET Identity Tables

ASP.NET Identity creates its own tables for users, roles, and claims:

- **AspNetUsers:** User accounts with email, password hash, and security stamp.
- **AspNetRoles:** The two application roles: Customer and Staff.
- **AspNetUserRoles:** Maps each user to their assigned role.
- **__EFMigrationsHistory:** Tracks applied migrations.

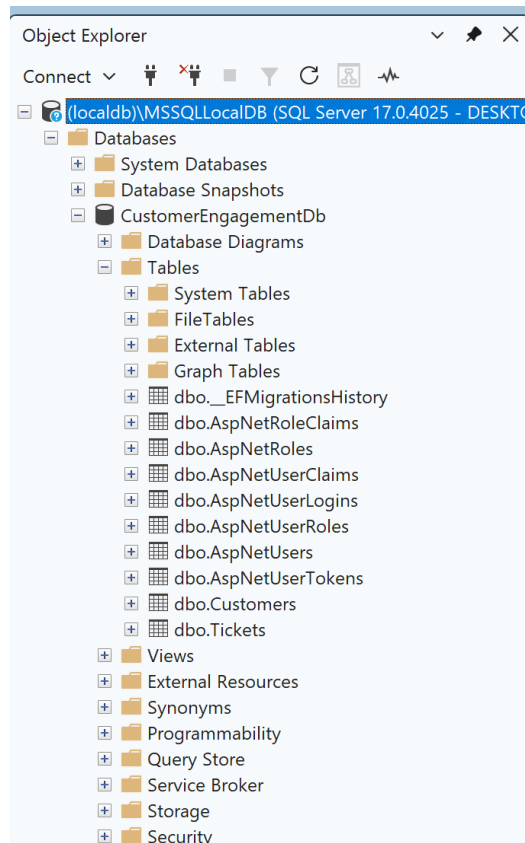


Figure 5: SQL Server Object Explorer - all tables created by EF Core migrations

Id	Name	Email	Phone	Address	CreatedAt
6	brug	user1@gmail.com	342342342342		2026-06-04 15:00:04.9168...
7	Ram mishra	user2@gmail.com	435345345435	8765	2026-06-04 15:01:59.0000...
8	bru2	user3@gmail.com	435345345435		2026-06-04 15:13:57.5151...
9	gggs	staff1@gmail.com	343454534543		2026-06-04 15:30:44.0779...
10	test 1 singh	test1@gmail.com	45675643232	4532	2026-06-04 16:01:32.0000...
11	Ajay singh	as@gmail.com	303932113	117.	2026-06-04 16:23:11.0000...
12	John Doe	customer@demo.com	+1 555-0199		2026-06-04 20:33:47.0301...
13	John singh	customerdemo1@gmail.com	+91 30393182...		2026-06-04 22:47:42.0481...

Figure 6: Customer table data in SQL Server

Id	Subject	Description	Status	Priority	AssignedAgent	CreatedAt	UpdatedAt
6	Api not working?	dufbdwbfdbfw	Resolved	High	Saumya	2026-06-04 15:00:04.0000000	2026-06-04 22:...
7	Schema mismatch	Blah blah	Resolved	Medium	Agent 1	2026-06-04 15:01:59.0000000	2026-06-04 22:...
8	Bruh	divaibfadbfbfblbfadbf	Resolved	Medium	Saumya	2026-06-04 15:13:57.0000000	2026-06-04 16:...
9	Sy5434	33345453454fgdfgdfg	Resolved	Low	Saumya	2026-06-04 15:30:44.0000000	2026-06-04 16:...
10	Bruh	some thing happened	Resolved	Low	Saumya	2026-06-04 15:35:20.0000000	2026-06-05 01:...
11	lagging	run a b c	Resolved	Medium	Saumya	2026-06-04 16:01:32.0000000	2026-06-04 22:...
12	lagging 2	more lagging	Resolved	High	Saumya	2026-06-04 16:02:03.0000000	2026-06-04 22:...
14	Idk man lets see	blah bal hbalh	Open	High	Agent 1	2026-06-04 17:14:19.0000000	2026-06-04 17:...
15	Test Ticket from Subagent	This is a test ticket description submitted via ...	Open	Medium		2026-06-04 20:33:47.0871685	NULL
16	Verified Minimal Brutalist UI	This ticket is created to verify the final minim...	Open	High		2026-06-04 20:36:32.9951119	NULL
17	Facing Lagging Issues	While opening the portal, I feel a strong lag.	Open	High		2026-06-04 22:47:42.1109627	NULL

Figure 7: Ticket table data in SQL Server

10. Entity Relationship Diagram

The ER diagram shows the relationships between the core entities stored in the database.

Relationship	Cardinality	Description
Customer to Ticket	1 : Many	One customer can have many tickets. Each ticket must belong to exactly one customer (FK enforced).
Customer toAspNetUsers	1 : 1	Each customer record links to one Identity user account through the UserId foreign key.
Ticket to Agent (AssignedTo)	Many : 1 (optional)	Many tickets can be assigned to one agent. Stored as a string field - no FK constraint.

ER Diagram

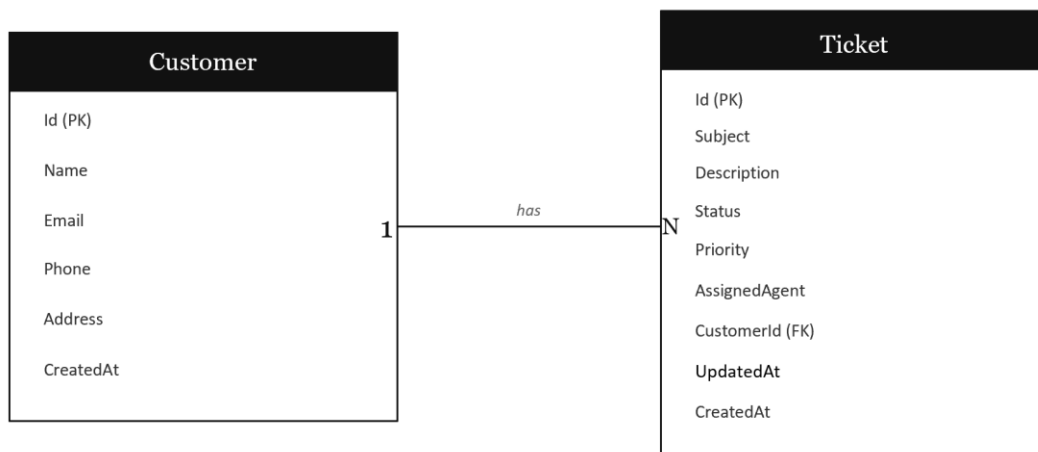


Figure 8: Entity Relationship Diagram

11. Technology Stack

Category	Technology	Version / Notes
Web Framework	ASP.NET Core MVC	.NET 8 - server-side rendering with Razor Views
API Framework	ASP.NET Core Web API	.NET 8 - REST endpoints returning JSON
Language	C#	C# 12 - primary application language
Authentication	ASP.NET Core Identity	Built-in user management with password hashing
API Security	JWT Bearer Authentication	Signed tokens with configurable expiry
Database	SQL Server (LocalDB)	Relational database for all application data
ORM	Entity Framework Core 8	Code-First approach with migrations
Frontend Styling	Bootstrap 5	Responsive UI components and grid
Frontend Scripting	jQuery / jQuery AJAX	AJAX ticket filtering on the staff tickets page
View Engine	Razor Pages / Views	Server-side HTML templating
API Documentation	Swagger / OpenAPI 3.0	Interactive API docs at /swagger
Containerization	Docker	Dockerfile for container build
CI/CD	GitHub Actions	Automated build and test on push
IDE	Visual Studio 2022	Development and debugging environment

12. Frontend and Backend Connectivity

The application uses two communication patterns between the frontend and backend.

12.1 Standard MVC Server-Side Rendering

The browser sends an HTTP request to an MVC Controller. The Controller calls the Service layer, which uses the Repository to query SQL Server through EF Core. The Controller builds a ViewModel and passes it to a Razor View. The View renders the full HTML page on the server.

12.2 AJAX Partial View Updates

On the Staff Tickets page, jQuery sends an AJAX GET request to a FilterTickets action method when any filter changes. The action queries the database with the filter parameters and returns a Partial View containing only the updated ticket table HTML. JavaScript replaces the ticket list container with the returned HTML. No full page reload occurs.

12.3 REST API Endpoints

The Web API layer is separate from the MVC layer and exists for programmatic access and Swagger testing. API requests require a JWT Bearer token in the Authorization header. The MVC frontend communicates with the database through the service and repository layers directly, not through the API.

13. Authentication and Authorization

13.1 ASP.NET Identity

User accounts are managed through ASP.NET Identity. On registration, email and password are stored in the AspNetUsers table. Passwords are hashed using PBKDF2 with HMAC-SHA256, including a per-user salt. Plain text passwords are never stored or compared.

13.2 Roles

- Customer: Assigned during customer registration. Grants access only to Customer Portal routes.
- Staff: Assigned to support staff accounts. Grants access to Dashboard, Customer management, and Ticket management.

13.3 Route Protection

```
[Authorize(Roles = "Staff")] applied to DashboardController,
CustomersController, TicketsController
[Authorize(Roles = "Customer")] applied to CustomerPortalController
```

Unauthenticated users are redirected to the login page. Users accessing unauthorized routes see the access denied page.

13.4 Login and Role Redirect

After successful login, the application checks the user's role and redirects accordingly. Staff are sent to the Dashboard; customers are sent to the Customer Portal. Neither type of user ever sees routes meant for the other role.

14. JWT Authentication Implementation

JWT (JSON Web Token) authentication secures the REST API layer, separate from the cookie-based session authentication used by the MVC views.

14.1 Step-by-Step Flow

10. Send a POST request to `/api/auth/login` with email and password in the JSON request body.
11. The API validates credentials against the ASP.NET Identity user store.
12. If valid, the server generates a JWT token containing the user email and role, signed with the secret key from `appsettings.json`.
13. The token is returned in the API response body.
14. Copy the token and paste it into the Swagger Authorize dialog.
15. All subsequent API requests from Swagger include this token in the Authorization header automatically.
16. The JWT middleware validates token signature and expiry on every incoming API request.

[illegible]

Figure 9: JWT token returned in the response body after POST /api/auth/login

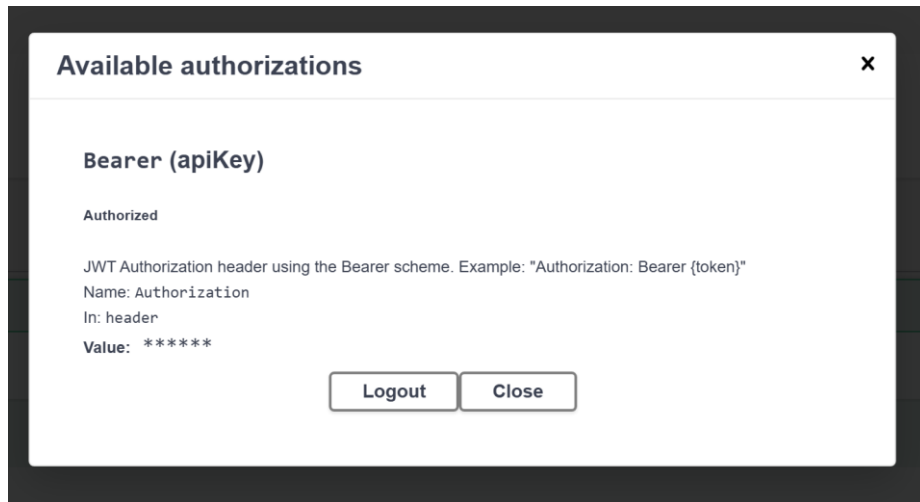


Figure 10: Swagger Authorize dialog with Bearer token active

14.2 Token Contents

- sub: User's email address
- role: The user's role (Customer or Staff)
- exp: Expiry timestamp
- iss and aud: Issuer and audience values configured in appsettings.json

15. Customer Management Module

The Customer Management module is available to Staff users only. It provides complete CRUD operations for managing customer records.

15.1 Features

- View a list of all customers with name, email, phone, and registration date.
- Add a new customer with first name, last name, email, phone, and address.
- Edit existing customer information.
- Delete a customer, subject to the active ticket business rule.

STAFF ACCESS

Customers

Manage customer records and review linked support ticket activity.

Create New Customer

NAME	EMAIL	PHONE	ADDRESS	CREATED AT	TICKETS	ACTIONS
brug	user1@gmail.com	342342342342		Jun 04, 2026	0	Edit Details Delete
Ram mishra	user2@gmail.com	435345345435	8765	Jun 04, 2026	0	Edit Details Delete
bruh2	user3@gmail.com	435345345435		Jun 04, 2026	2	Edit Details Delete
gggs	staff1@gmail.com	343454534543		Jun 04, 2026	2	Edit Details Delete
test 1 singh	test1@gmail.com	45675643232	4532	Jun 04, 2026	0	Edit Details Delete
test 2 singh	test2@gmail.com	45675643232	4532	Jun 04, 2026	0	Edit Details Delete

Figure 11: Customer management page - customer list with edit and delete options

15.2 Business Rule: Active Ticket Check

Before a customer record can be deleted, the application checks whether any of their tickets have a status of Open or In Progress. If such tickets exist, deletion is blocked and the staff member sees an error message. This ensures no customer with unresolved tickets can be accidentally removed.

15.3 Validation

Customer forms use [Required], [EmailAddress], and [MaxLength] validation attributes. Validation errors are shown inline on the form. Server-side validation runs in the controller independently of client-side checks.

16. Ticket Management Module

The Ticket Management module allows staff to create, view, update, assign, and resolve support tickets.

16.1 Features

- View all tickets in a searchable and filterable list.
- Create a new ticket on behalf of a customer.
- Edit ticket title, description, and priority.
- Update ticket status (Open, In Progress, Resolved).
- Assign a ticket to a staff agent.
- Delete a ticket.

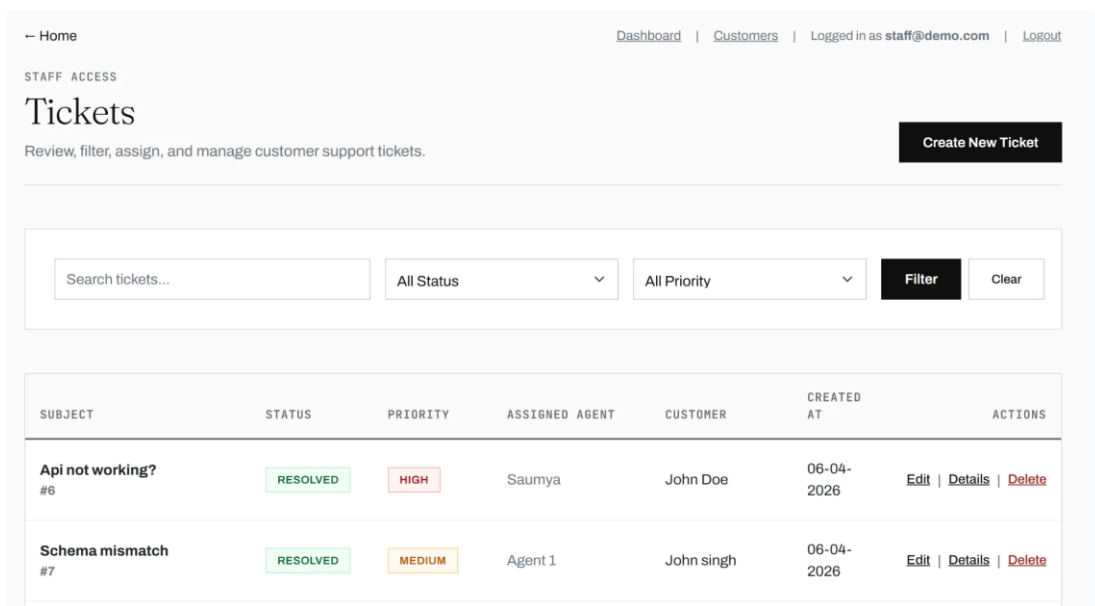


Figure 12: Ticket management page - list with status badges, priority badges, and action links

16.2 Status Workflow

All new tickets start with Open status. Staff move them through the workflow:

Open --> In Progress --> Resolved

16.3 Priority Levels

Priority	Badge Color	Intended Use
High	Red	Critical issues needing immediate attention.
Medium	Orange	Standard issues to address in normal course.
Low	Green	Minor requests or non-critical feedback.

17. Dashboard Module

The Staff Dashboard is the first page staff see after logging in. It gives an overview of current customer and ticket metrics.

17.1 Stat Cards

Card	What It Shows
Total Customers	Count of all customer records in the Customers table
Total Tickets	Total count of all tickets regardless of status
Open Tickets	Count of tickets where Status = Open
Resolved Tickets	Count of tickets where Status = Resolved
Unassigned Tickets	Count of tickets where AssignedTo field is empty

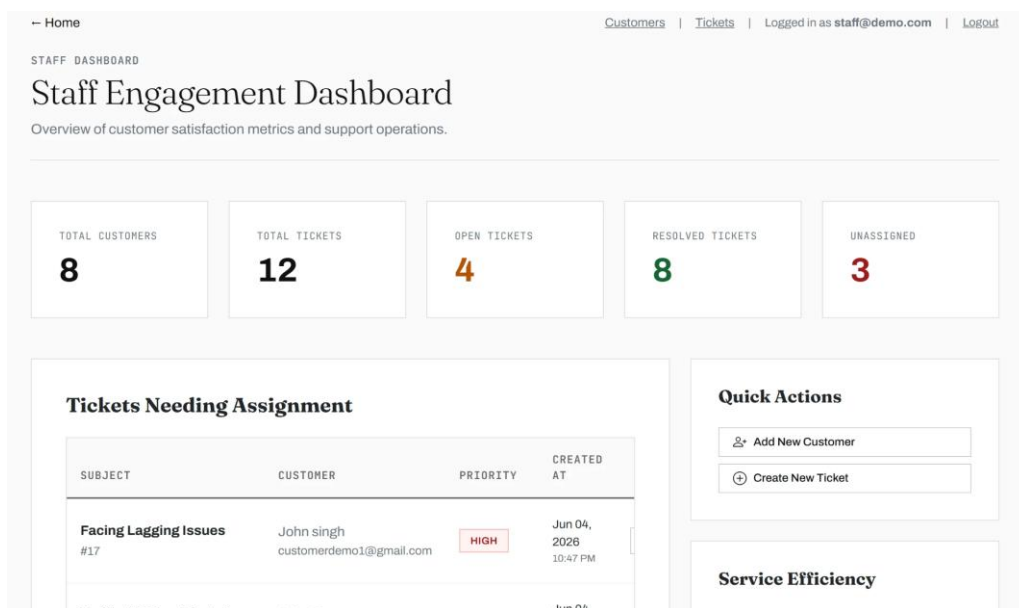


Figure 13: Staff Dashboard - stat cards showing current metrics

17.2 Tickets Needing Assignment

Below the stat cards, the dashboard shows a table of tickets not yet assigned to a staff agent. This helps staff quickly identify work that needs to be picked up.

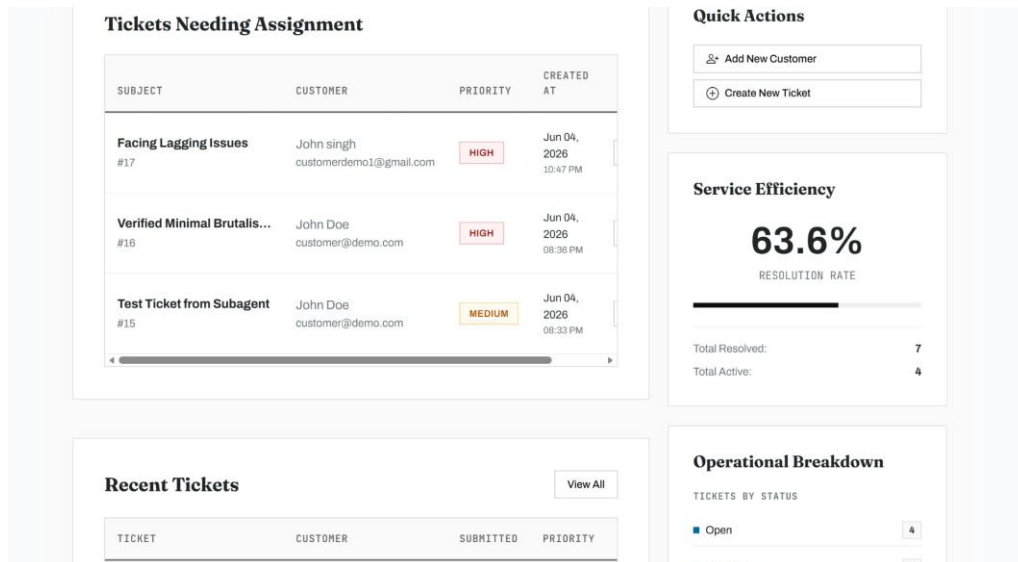


Figure 14: Dashboard - unassigned tickets panel and service efficiency metric

17.3 Service Efficiency

A Service Efficiency percentage shows the number of resolved tickets divided by total tickets. This gives a quick snapshot of how well the team is keeping up with incoming requests.

18. AJAX Filtering Implementation

The tickets page uses AJAX filtering so staff can search and filter without waiting for a full page reload.

18.1 Filter Controls

- Search box: Filters tickets by title or description text.
- Status dropdown: Shows only Open, In Progress, or Resolved tickets.
- Priority dropdown: Shows only Low, Medium, or High priority tickets.

All three filters work together. Staff can search for a word and filter by status and priority simultaneously.

The screenshot shows a web application interface for managing tickets. At the top, there's a navigation bar with links to Home, Dashboard, Customers, and a login status for 'staff@demo.com'. Below this, the 'STAFF ACCESS' section is titled 'Tickets' with a subtitle 'Review, filter, assign, and manage customer support tickets.' and a 'Create New Ticket' button. A filter bar contains a search input with 'lag', a status dropdown set to 'Resolved', a priority dropdown set to 'All Priority', and 'Filter' and 'Clear' buttons. Below the filter bar is a table of tickets.

SUBJECT	STATUS	PRIORITY	ASSIGNED AGENT	CUSTOMER	CREATED AT	ACTIONS
lagging #11	RESOLVED	MEDIUM	Saumya	bruh2	06-04-2026	Edit Details Delete
lagging 2 #12	RESOLVED	HIGH	Saumya	John Doe	06-04-2026	Edit Details Delete

Figure 15: AJAX filtering in action - search 'lag' with Resolved status filter applied

18.2 Technical Implementation

When any filter changes, a jQuery event handler intercepts the form submission and calls `jQuery.ajax()` to send a GET request to the `FilterTickets` action method with filter values as query parameters.

The action applies filters to the EF Core query (WHERE clauses at the database level) and returns a Partial View containing only the ticket table HTML. jQuery replaces the ticket list container with the returned HTML. No full page reload occurs.

18.3 Why This Matters

Without AJAX, every filter change would reload the entire page HTML. With AJAX, only the ticket list rows update. This makes filtering noticeably faster and keeps filter values visible between requests.

19. Repository Pattern Implementation

All database access in the application goes through the Repository Pattern. No controller or service class calls EF Core directly.

19.1 Structure

Component	Role
ITicketRepository	Interface defining all ticket data operations: GetAll, GetById, GetByCustomerId, Create, Update, Delete, GetUnassigned, CountByStatus.
TicketRepository	Implements ITicketRepository using ApplicationDbContext and EF Core LINQ queries.
ICustomerRepository	Interface defining all customer operations: GetAll, GetById, Create, Update, Delete, HasActiveTickets.
CustomerRepository	Implements ICustomerRepository using ApplicationDbContext.

19.2 Benefits

- The service layer calls ITicketRepository methods without knowing whether data comes from SQL Server or a mock. This enables unit testing.
- All EF Core code is in one place. If the ORM changes, only repository implementations need updating.
- Separation of concerns is maintained: repositories handle data access, services handle business rules.

19.3 Registration

```
builder.Services.AddScoped<ITicketRepository, TicketRepository>();  
builder.Services.AddScoped<ICustomerRepository, CustomerRepository>();
```

20. Dependency Injection

ASP.NET Core has a built-in Dependency Injection container. This application registers all services and repositories in Program.cs and injects them through constructor parameters.

20.1 What Is Registered

```
builder.Services.AddScoped<ITicketRepository, TicketRepository>();
builder.Services.AddScoped<ICustomerRepository, CustomerRepository>();
builder.Services.AddScoped<ITicketService, TicketService>();
builder.Services.AddDbContext<ApplicationDbContext>(...);
builder.Services.AddIdentity<ApplicationUser, IdentityRole>()...;
```

20.2 Constructor Injection

The TicketsController receives its dependencies through its constructor:

```
public TicketsController(ITicketRepository ticketRepository, ICustomerRepository
customerRepository)
```

The DI container automatically provides the correct implementations when the controller is instantiated. The controller never calls new TicketRepository() directly.

20.3 Service Lifetime

Repositories and services are registered with Scoped lifetime: one instance per HTTP request. This is correct for EF Core DbContext and its dependencies.

21. SOLID Principles

Principle	How Applied in This Project
Single Responsibility (SRP)	TicketsController handles only ticket HTTP requests. TicketRepository handles only ticket data access. TicketService handles only ticket business logic. Each class has one reason to change.
Open / Closed (OCP)	ITicketRepository can be given a new implementation without modifying TicketService or any controller. The interface is open for extension, closed for modification.
Liskov Substitution (LSP)	Any class implementing ITicketRepository can replace TicketRepository without breaking dependent code. Demonstrated by mock implementations in unit tests.
Interface Segregation (ISP)	Separate interfaces for ticket and customer repositories. Controllers and services only receive the interface they actually need.
Dependency Inversion (DIP)	High-level modules (Controllers, Services) depend on abstractions (ITicketRepository) not concrete classes. The DI container wires up instances at runtime.

22. Design Patterns Used

Pattern	Where Applied	Purpose
MVC Pattern	Entire ASP.NET Core MVC application	Separates data model, views, and application logic into independent layers.
Repository Pattern	Data access layer (ITicketRepository, ICustomerRepository)	Abstracts EF Core queries behind interfaces for testability and separation of concerns.
Dependency Injection	Program.cs, all controllers and services	Manages object creation and lifecycle through the built-in ASP.NET Core DI container.
ViewModel Pattern	MVC controllers and Razor Views	Dedicated ViewModel classes shape data for each view, preventing over-posting and direct entity exposure.

23. Web API Documentation

The REST API layer exposes endpoints for authentication, customer data, and ticket data. All endpoints except login require a JWT Bearer token.

Method	Endpoint	Description	Auth
POST	/api/auth/login	Accepts email and password JSON body. Returns JWT token on success.	None
GET	/api/customers	Returns a JSON array of all customer records.	JWT Required
GET	/api/customers/{id}	Returns a single customer by CustomerId.	JWT Required
GET	/api/tickets	Returns a JSON array of all ticket records.	JWT Required
GET	/api/tickets/{id}	Returns a single ticket by TicketId.	JWT Required

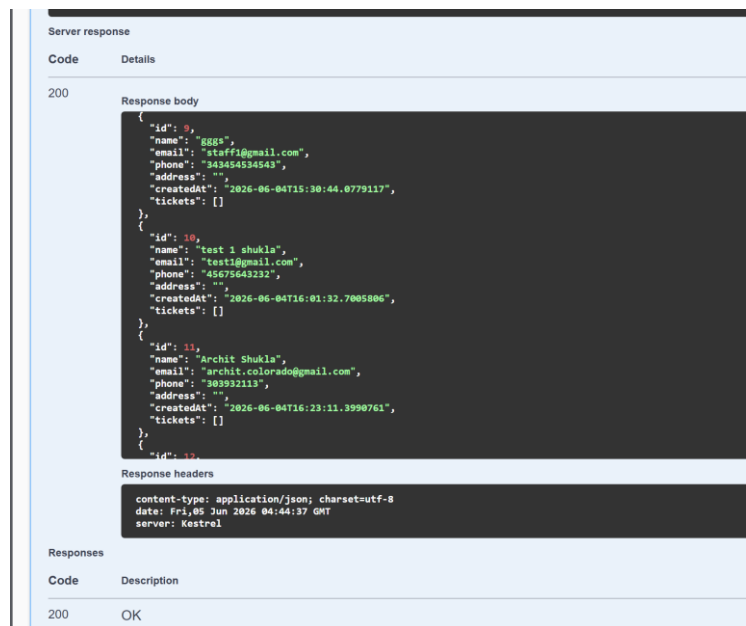


Figure 16: Swagger test result - GET /api/customers returning 200 OK with customer JSON array

23.1 Request and Response Format

All API endpoints accept and return JSON. The Authorization header for protected endpoints must contain: Bearer {token} where the token is the JWT string returned by the login endpoint.

24. Swagger Integration

Swagger (OpenAPI 3.0) is configured in the application to provide interactive API documentation accessible at /swagger in development mode.

24.1 Swagger UI

The Swagger UI shows all API endpoints grouped by controller (AuthApi, CustomersApi, TicketsApi). Each endpoint can be expanded to see the request format, parameters, and example responses.

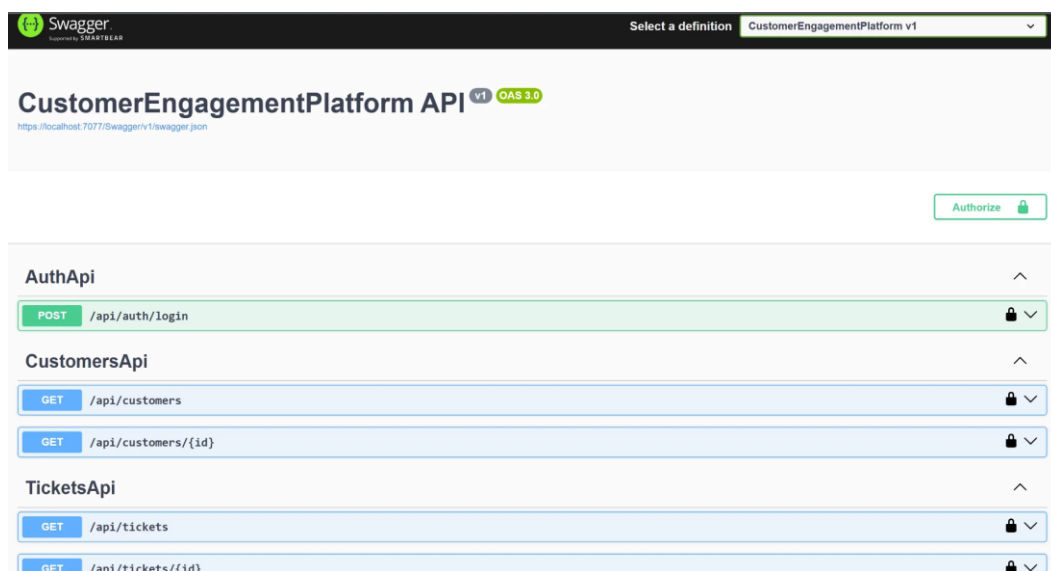


Figure 17: Swagger UI showing all three API groups

24.2 JWT Authorization

The Swagger UI includes an Authorize button. After calling the login endpoint to get a token, paste it into the Authorize dialog. Swagger then includes it automatically in the Authorization header for all subsequent requests.

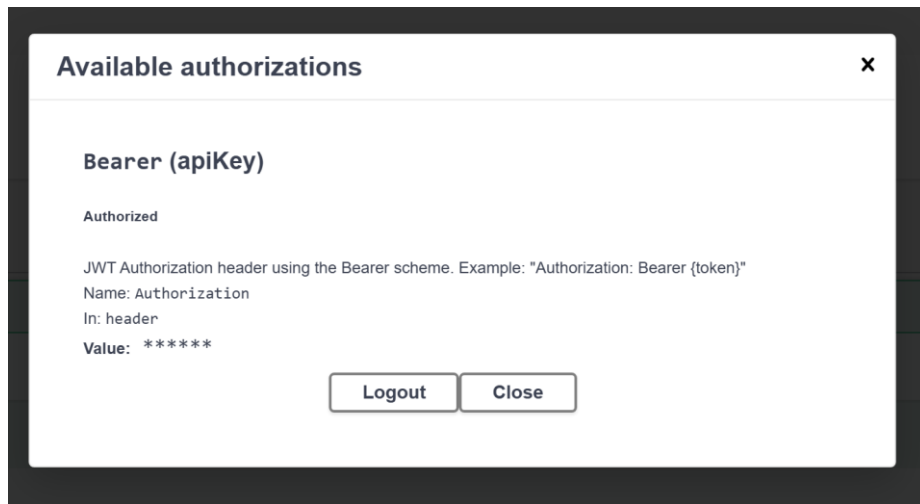


Figure 18: Swagger Authorize dialog with Bearer token active

24.3 Configuration

Swagger is configured in Program.cs with a SecurityDefinition for BearerAuth and SecurityRequirements applied so protected endpoints show the lock icon in the UI.

25. Business Rules Summary

Rule ID	Business Rule	Enforcement Location
BR-01	Only authenticated users can access protected pages.	[Authorize] attributes on all restricted controllers.
BR-02	Customers can only access Customer Portal routes.	[Authorize(Roles='Customer')] on CustomerPortalController.
BR-03	Staff can only access Dashboard and management routes.	[Authorize(Roles='Staff')] on Dashboard, Customers, Tickets controllers.
BR-04	All new tickets are created with Open status.	Set in the ticket creation service method. Not changeable through creation form.
BR-05	Ticket status follows the defined workflow.	Status values constrained to Open, In Progress, Resolved in the edit form.
BR-06	A customer cannot be deleted if they have active tickets.	CustomerService.DeleteCustomer() checks for Open or In Progress tickets first.
BR-07	REST API endpoints require a valid JWT token.	JWT middleware validates tokens on every API request. Returns 401 if invalid.
BR-08	Ticket ownership is linked to a customer account.	CustomerId is a required FK on the Ticket table. EF Core enforces referential integrity.

26. Validation and Security Measures

26.1 Input Validation

All forms use ASP.NET Core Model Validation with Data Annotation attributes. The controller checks `ModelState.IsValid` before processing. Validation errors are shown inline on the form. Server-side validation runs independently of client-side checks.

26.2 Password Security

ASP.NET Identity hashes all passwords using PBKDF2 with HMAC-SHA256 and a per-user random salt. Passwords are never stored, logged, or compared in plain text.

26.3 SQL Injection Prevention

Entity Framework Core uses parameterized queries for all database operations. No raw SQL strings are built from user input. LINQ queries are translated to parameterized SQL by EF Core.

26.4 JWT Token Security

JWT tokens are signed using HMAC-SHA256 with a secret key stored in `appsettings.json`. Tokens have a configured expiry time. The middleware validates signature, issuer, audience, and expiry on every API request.

26.5 Route Protection

The `[Authorize]` attribute is applied at the class level on all restricted controllers. New action methods added to the controller automatically inherit the authorization requirement.

27. Testing and Verification

27.1 Unit Testing

Unit tests were written for the service and repository layers using xUnit. Tests cover:

- Ticket creation: Verifies new tickets are always assigned Open status.
- GetById and GetAll: Tests return expected results.
- Customer deletion check: Verifies active ticket check blocks deletion correctly.
- Status updates: Tests that updated status is persisted.

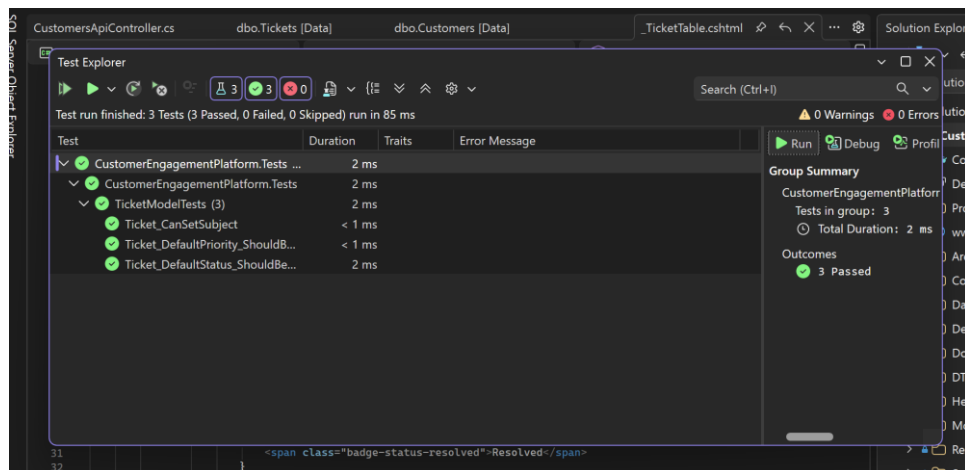


Figure 19: Unit test results

27.2 Role-Based Access Testing

Manual testing confirmed all role restrictions:

- Customer accounts cannot access /Dashboard, /Customers, or /Tickets. Access denied page shown.
- Staff accounts cannot access Customer Portal routes.
- Unauthenticated access to all protected routes redirects to the login page.

27.3 API Testing via Swagger

- POST /api/auth/login with valid credentials returns 200 with JWT token.
- POST /api/auth/login with wrong password returns 401.
- GET /api/customers without token returns 401 Unauthorized.
- GET /api/customers with valid token returns 200 with customer array.
- Expired token tested: returns 401.

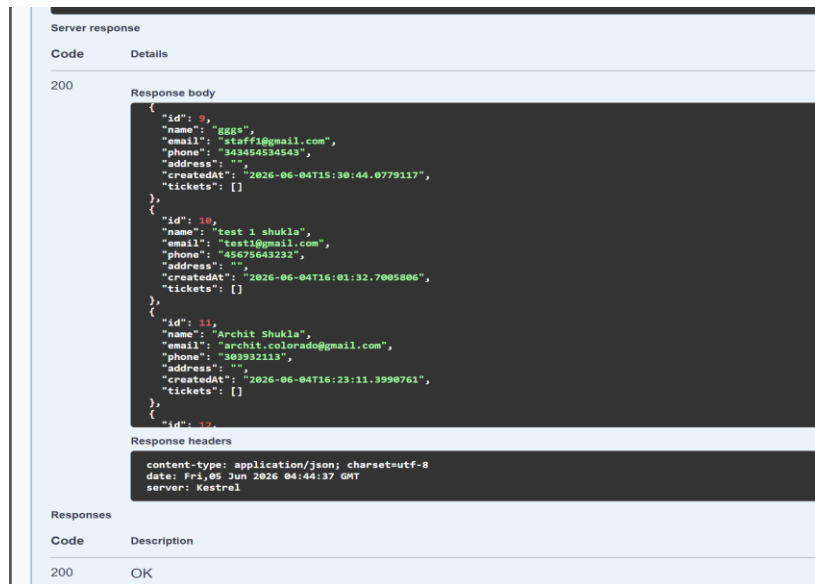


Figure 20: Swagger test - GET /api/customers returning 200 OK

27.4 Database Verification

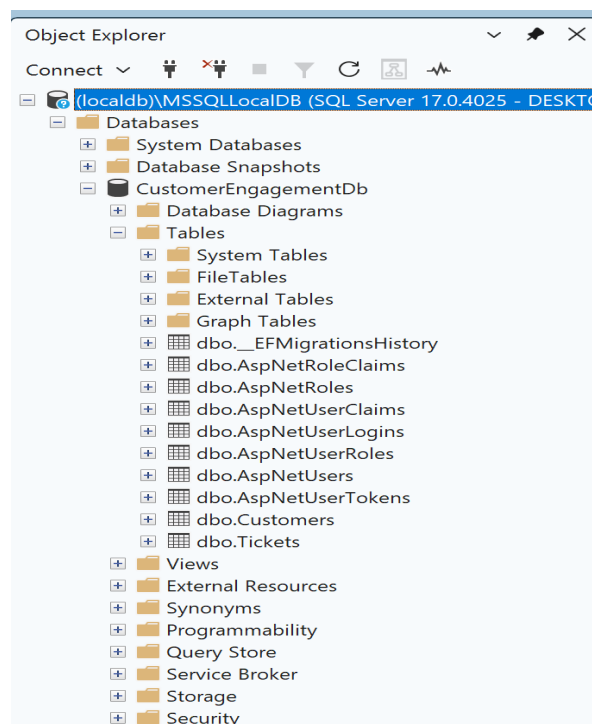


Figure 21: SQL Server Object Explorer - all tables created by EF Core migrations

27.5 CRUD End-to-End Testing

- Customer creation, edit, and delete operations verified through the staff UI.
- Ticket creation, status update, and agent assignment verified with actual database records.
- Customer Deletion block verified by attempting to delete a customer with an open ticket.

28. DevOps Assets

Two DevOps configuration files were created as part of the project.

28.1 Dockerfile

The Dockerfile uses a two-stage build:

17. Stage 1 (Build): Uses the .NET SDK image to restore packages, compile, and publish the output.
18. Stage 2 (Runtime): Uses the lighter ASP.NET runtime image and copies only the published output from Stage 1. The SDK is not included in the final image, keeping the image size minimal.

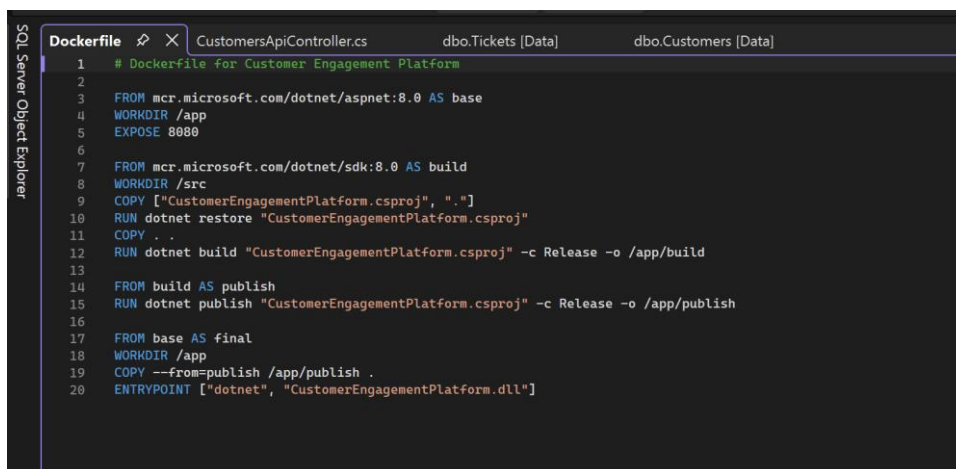


Figure 22: Dockerfile using multi-stage build

28.2 GitHub Actions Workflow

A GitHub Actions YAML workflow file in .github/workflows/ triggers on every push to main and on all pull requests.

Workflow steps:

19. Checkout repository code.
20. Set up .NET 8 SDK environment.
21. Run dotnet restore to install NuGet packages.
22. Run dotnet build to compile and check for errors.
23. Run dotnet test to execute all unit tests.

```

.github > workflows > Y clyml
1  name: Customer Engagement Platform CI
2
3  on:
4    push:
5      branches:
6        - main
7
8    pull_request:
9      branches:
10       - main
11
12  jobs:
13    build-test:
14
15      runs-on: ubuntu-latest
16
17      steps:
18        - name: Checkout code
19          uses: actions/checkout@v4
20
21        - name: Setup .NET
22          uses: actions/setup-dotnet@v4
23          with:
24            dotnet-version: '8.0.x'
25
26        - name: Restore packages
27          run: dotnet restore
28
29        - name: Build solution
30          run: dotnet build --no-restore
31
32        - name: Run tests
33          run: dotnet test --no-build
34

```

Figure 23: GitHub Actions workflow YAML configuration

29. Comparison With Real-World Applications

The Customer Engagement Platform addresses the same core problem as commercial tools like Zendesk and Freshdesk. This section shows what was built versus what those enterprise platforms offer.

Feature Area	This Application	Zendesk / Freshdesk
Ticket Management	Full CRUD with Open, In Progress, Resolved workflow	Configurable workflows with custom statuses, SLA timers, escalation rules
Customer Portal	Web portal for raising and tracking tickets	Multi-channel: web, email, chat, phone, social media
Role-Based Access	Two roles: Customer and Staff	Granular roles: agent, team lead, admin, viewer
Dashboard	Basic stat cards: total, open, resolved, unassigned	Advanced analytics with charts, SLA reports, agent performance metrics
REST API	5 endpoints with JWT auth and Swagger docs	Full API with webhooks, rate limiting, and API key management
Search and Filter	AJAX filtering by title, status, priority	Full-text search across all fields with custom field filtering
Email Notifications	Not implemented - future scope	Automatic notifications on all ticket events
AI Features	Not implemented - future scope	AI summarization, suggested responses, sentiment analysis
Cloud Deployment	Dockerfile only	Fully hosted SaaS with global CDN
Mobile App	Not implemented	Native iOS and Android apps

The core ticket management workflow, role separation, dashboard, and REST API are all implemented and working. Features like AI summarization, email notifications, cloud deployment, and multi-channel support are enterprise-scale additions noted as future scope only.

30. Assumptions and Limitations

30.1 Assumptions Made

- The application is designed for a small support team. No multi-team or department structure is implemented.
- Staff accounts are created manually. No public registration path exists for staff to prevent unauthorized account creation.
- The application runs on localhost with SQL Server LocalDB for development.
- Email addresses are assumed to be unique across the entire system.

30.2 Known Limitations

- No email notification system. Customers must log in to check ticket status manually.
- The GitHub Actions workflow runs build and test steps but does not deploy automatically.
- The JWT secret key is in appsettings.json. In production this should be in a secrets manager.
- The ticket AssignedTo field stores the agent name as a plain string with no FK validation.
- No file attachment support for tickets.
- Pagination is not consistently applied across all list views.

31. Best Practices Followed

Best Practice	How Applied
Layered Architecture	Presentation, API, Service, Repository, and Database layers are clearly separated. No layer bypasses the one directly below it.
Repository Pattern	All EF Core access is abstracted behind IRepository interfaces. No controller or service calls ApplicationDbContext directly.
Dependency Injection	All services and repositories are constructor-injected through the ASP.NET Core DI container registered in Program.cs.
Role-Based Authorization	Every restricted route uses [Authorize(Roles='...')] at the class level. Default access is denied for all protected controllers.
JWT Security	API endpoints are protected with signed, time-limited JWT tokens. Middleware validates signature, issuer, audience, and expiry on every request.
Input Validation	Data Annotations validate all incoming form data at the controller level. Server-side validation runs independently of client-side checks.
EF Core Migrations	All schema changes are tracked through EF Core code-first migrations. Running the update command keeps database in sync with the C# model.
ViewModel Pattern	Dedicated ViewModels are used between controllers and views. Domain entities are not directly bound to view forms, preventing over-posting.
No Raw SQL	All queries use EF Core LINQ with parameterized inputs. No raw SQL strings are built from user input.
Single Responsibility	Each class has one clear purpose. Controller handles HTTP, service handles business rules, repository handles data access.

32. Future Scope

Feature	Description
Email Notifications	Send automatic emails to customers when ticket status changes or an agent replies. Could use SendGrid or SMTP.
AI Ticket Summarization	Integrate an AI API to generate short summaries of long ticket descriptions for staff.
Agent Workload Balancing	Auto-assign new tickets to the agent with the fewest open tickets.
Cloud Deployment	Deploy to Azure App Service with Azure SQL Database and environment-based production settings.
Advanced Analytics	Add charts and trend graphs for ticket volume, average resolution time, and agent performance.
Ticket Comments	Allow customers and staff to post replies within a ticket, creating a conversation thread.
File Attachments	Allow customers to attach screenshots or documents when raising a ticket.
Real-Time Notifications	Use SignalR to push notifications to staff when new tickets arrive.
Admin Role	A third role for administrators to manage staff accounts and configure system settings.
Mobile Optimization	Further improve the responsive design for mobile devices.

33. Conclusion

The Customer Engagement Platform capstone project successfully delivered a working multi-user web application built from scratch using technologies covered during the Wipro training program.

The application provides a functional Customer Portal for raising and tracking support tickets, a Staff Dashboard with live statistics, a complete ticket management workflow, JWT-secured REST API endpoints with Swagger documentation, AJAX-based live filtering, and DevOps configuration files including a Dockerfile and GitHub Actions workflow.

The codebase follows the Repository Pattern, Dependency Injection, SOLID principles, and a clean layered architecture throughout. Unit tests cover the service and repository layers, and all functionality was verified through manual testing and Swagger.

The comparison with tools like Zendesk and Freshdesk shows that the core concepts are correctly implemented. The gap is in enterprise-scale features like AI, email notifications, and cloud deployment, which are noted as future scope. This reflects a realistic and honest scope for a training project.

Building this project provided end-to-end experience across system design, database modeling, backend API development, frontend MVC development, authentication, security, testing, and DevOps. These are skills and patterns that apply directly to professional ASP.NET Core development.

--- End of Report ---

Saumya Singh | Wipro Training Program | June 2026